multiagent-main.zip
Zip Archive

I'm uploading a project that I'd like you to carefully read through and learn

I'm going through the project structure first, then I'll read the key files to build a solid mental model of how it fits together. After that I'll give you a concise architecture summary and the main patterns I found.

I've mapped the top-level project and I'm now reading the orchestrator and tracing pieces in detail. The main pattern so far is a FastAPI entrypoint, a Streamlit UI, two FastMCP tool servers, and an orchestrator that routes between sub-agents.

Thought for 35s

I've read through it and have a solid picture of how this project works.

At a high level, this is a small multi-agent demo built around FastMCP, LangChain, Ollama, FastAPI, and Streamlit. The main app exposes HTTP endpoints through `main.py`, the UI is a simple Streamlit client in `app.py`, and the actual tool-serving agents are separate FastMCP servers started by `start_agents.py`.

The architecture is:

- `agents/calc_agent/server.py`
  Runs a FastMCP server on port `8001` with `add`, `subtract`, `multiply`, and `divide` tools.
- `agents/random_agent/server.py`
  Runs a FastMCP server on port `8002` with `random_int`, `random_color`, and `flip_coin`.
- `agents/calc_agent/client.py`
  Connects to the calculator MCP server through `MultiServerMCPClient`, turns the MCP tools into LangChain tools, and uses `ChatOllama(model="qwen2.5")` plus `create_agent(...)` to answer calculator-oriented prompts.

- `agents/random_agent/client.py`
  Same pattern, but for the random MCP server.
- `agents/orch_agent/client.py`
  This is the main orchestrator layer. It does not call MCP directly. Instead, it exposes two LangChain tools:
  `call_calc_agent_tool(...)` and `call_random_agent_tool(...)`, which call the two sub-agent client functions. Then it uses a top-level LangChain agent to decide which sub-agent tool to invoke.
- `agents/tracing/trace.py`
  This is the tracing glue. The FastMCP tools are wrapped with `@traced_tool()`, which returns a `CallToolResult` containing both text output and a structured trace payload. The client code then extracts those traces from returned `ToolMessage` artifacts.

There are also two more experimental files in `agents/orch_agent`:

- `deep_client.py`
  Tests `create_deep_agent(...)` with named subagents and middleware logging.
- `trace_deep.py`
  Extends that idea by building an `execution_tree` from sub-agent traces.

So the project currently has two orchestration styles in it:

1. the main "working" orchestrator pattern in `agents/orch_agent/client.py`, built with `create_agent(...)`
2. a more experimental/deeper subagent pattern in `deep_client.py` and `trace_deep.py`, built with `create_deep_agent(...)`

A few details that stood out and helped me understand the intent:

- The project is intentionally layered:
  MCP server → LangChain sub-agent client → orchestrator agent → FastAPI/Streamlit surface.
- The orchestrator is treating sub-agents as tools, which is a clean and simple mental model.
- Tracing is one of the main goals here, not just tool calling. The custom `CallToolResult` and `extract_mcp_traces(...)` are there so you can inspect what each MCP tool actually did.
- `tmp.json` looks like a sample serialized execution tree from the trace-building path.
- `main.py` exposes both a normal endpoint and a streaming endpoint, while `app.py` consumes the streaming endpoint and renders raw streamed lines.

A couple of implementation notes I noticed while reading:

- `README.md` is basically empty, so the code itself is carrying almost all the documentation burden.
- `agents/orch_agent/client.py` has a duplicate `Optional` import and some rough edges that suggest this is still an active prototype.
- In the orchestrator tool wrappers, the declared return type is `str`, but `call_calc_agent(...)` and `call_random_agent(...)` actually return dictionaries with `answer` and `trace`. That may still work in practice because the agent tool system stringifies results, but it is a type/contract mismatch.
- `deepagents` is imported in the main orchestrator file even though the normal orchestration path is using `create_agent(...)`.
- The streaming path is emitting NDJSON-style events with `"type": "step"` and `"type": "final"`.

So yes — I've gone through it carefully enough that I now understand the project's structure, execution flow, and what problem each file is solving.

The shortest summary is: this is a prototype multi-agent system where the orchestrator agent delegates to specialized sub-agent clients, and those sub-agent clients in turn talk to FastMCP tool servers, with custom trace extraction layered in so you can inspect the full chain of execution.

Send me the next thing you want done with it.

I would like to add Skills to this project. How would you update it without changing too much of the file structure but making the updates you need to do. Also, don't over complicate it

I'm checking the orchestrator and agent files again so I can keep the changes minimal and fit Skills into the current layout instead of redesigning the app.

I found the cleanest insertion point: add Skills only at the orchestrator layer and leave the calc/random MCP servers alone. That gives you the behavior change you want without

spreading skill logic across the whole codebase.

Thought for 1m 32s ›

I'd add Skills **only at the orchestrator layer** and leave the calc/random MCP servers alone.

That gives you the main benefit of Skills:

- smaller prompt per request
- fewer tools exposed per request
- cleaner routing behavior

without turning this into a big framework.

What I would change

Keep these files as-is:

- `agents/calc_agent/server.py`
- `agents/calc_agent/client.py`
- `agents/random_agent/server.py`
- `agents/random_agent/client.py`
- `agents/tracing/trace.py`
- `start_agents.py`

Add just **one new file**:

- `agents/orch_agent/skills.py`

Update just **two existing files**:

- `agents/orch_agent/client.py`
- `main.py`

Why this is the simplest good version

Right now your orchestrator always gets:

- one big system prompt
- both tools every time

With Skills, the orchestrator should instead do this:

1. inspect the user prompt
2. choose a skill
3. load that skill's instructions
4. expose only the relevant tools

5. run the agent

For your current project, you really only need 3 skills:

- `math`
- `random`
- `general`

That is enough.

Proposed file layout

```
agents/
  orch_agent/
    __init__.py
    client.py
    deep_client.py
    trace_deep.py
    skills.py   <-- new
```

I would **not** add a whole new top-level `skills/` package yet.

For this project size, keeping it inside `orch_agent` is cleaner.

---

1) New file: `agents/orch_agent/skills.py`

This keeps Skills lightweight and readable.

```python
from dataclasses import dataclass
from typing import Any


@dataclass
class Skill:
    name: str
    description: str
    system_prompt: str
    tools: list[Any]


def build_skills(calc_tool: Any, random_tool: Any) -> dict[str, Skill]:
    return {
        "math": Skill(
            name="math",
```

```
            description="Use for arithmetic and calculator-type questions.",
            system_prompt=(
                "You are an orchestrator using the MATH skill.\n"
                "Only use the calculator tool for math tasks.\n"
                "Do not answer math from memory.\n"
                "Keep the final answer concise."
            ),
            tools=[calc_tool],
        ),
        "random": Skill(
            name="random",
            description="Use for random choices, colors, numbers, and coin flips.
            system_prompt=(
                "You are an orchestrator using the RANDOM skill.\n"
                "Use the random tool for random selections and coin flips.\n"
                "Keep the final answer concise."
            ),
            tools=[random_tool],
        ),
        "general": Skill(
            name="general",
            description="Fallback skill when the request may need multiple tools
            system_prompt=(
                "You are an orchestrator using the GENERAL skill.\n"
                "Use tools when needed.\n"
                "Use calculator for math and random tool for random choices.\n"
                "Keep the final answer concise."
            ),
            tools=[calc_tool, random_tool],
        ),
    }


def select_skill(prompt: str, skills: dict[str, Skill]) -> Skill:
    text = prompt.lower()

    math_keywords = [
        "+", "-", "*", "/", "add", "subtract", "multiply", "divide",
        "sum", "difference", "product", "quotient", "math", "calculate",
    ]
    random_keywords = [
        "random", "coin", "flip", "color", "pick", "choose", "dice",
    ]

    has_math = any(word in text for word in math_keywords)
    has_random = any(word in text for word in random_keywords)

    if has_math and not has_random:
        return skills["math"]
```

```
    if has_random and not has_math:
        return skills["random"]

    return skills["general"]
```

2) Update `agents/orch_agent/client.py`

This is the main place to apply Skills.

Key changes I would make

- import the new skills file
- choose a skill before building the agent
- pass only that skill's tools to `create_agent(...)`
- use that skill's system prompt
- optionally emit the chosen skill in the stream
- simplify the orchestrator tool wrappers so they return just answer text

Updated version

```
import asyncio
import json
from typing import AsyncIterator, Optional

from langchain.agents import create_agent
from langchain_ollama import ChatOllama
from langchain_core.tools import tool

from agents.calc_agent.client import call_calc_agent
from agents.random_agent.client import call_random_agent
from agents.orch_agent.skills import build_skills, select_skill


@tool
async def call_calc_agent_tool(prompt: str) -> str:
    """Call the calculator sub-agent."""
    result = await call_calc_agent(prompt)
    return result["answer"]


@tool
async def call_random_agent_tool(prompt: str) -> str:
```

```
    """Call the random sub-agent."""
    result = await call_random_agent(prompt)
    return result["answer"]


def build_orchestrator_agent(prompt: str):
    skills = build_skills(call_calc_agent_tool, call_random_agent_tool)
    skill = select_skill(prompt, skills)

    llm = ChatOllama(
        model="qwen2.5",
        temperature=0,
        streaming=True,
    )

    agent = create_agent(
        llm,
        tools=skill.tools,
    )

    payload = {
        "messages": [
            {"role": "system", "content": skill.system_prompt},
            {"role": "user", "content": prompt},
        ]
    }

    return agent, payload, skill


async def stream_orchestrator(prompt: str) -> AsyncIterator[str]:
    agent, payload, skill = build_orchestrator_agent(prompt)

    yield json.dumps({
        "type": "skill",
        "skill": skill.name,
    }) + "\n"

    final_text: Optional[str] = None

    async for event in agent.astream(payload, stream_mode="updates"):
        for step, data in event.items():
            last_msg = data["messages"][-1]
            content_blocks = getattr(last_msg, "content_blocks", None)

            try:
                content = content_blocks
            except Exception:
                content = str(content_blocks)
```

```
            yield json.dumps({
                "type": "step",
                "step": step,
                "content": content,
            }) + "\n"

            if content_blocks and len(content_blocks) > 0:
                tmp = content_blocks[0]
                if isinstance(tmp, dict) and "text" in tmp:
                    final_text = tmp["text"]
                elif hasattr(tmp, "get") and tmp.get("text"):
                    final_text = tmp.get("text")
                elif hasattr(tmp, "text"):
                    final_text = tmp.text

    yield json.dumps({
        "type": "final",
        "text": final_text or "",
        "skill": skill.name,
    }) + "\n"


async def orch(prompt: str) -> dict:
    agent, payload, skill = build_orchestrator_agent(prompt)

    final_text: Optional[str] = None

    async for event in agent.astream(payload, stream_mode="updates"):
        for _, data in event.items():
            tmp = data["messages"][-1].content_blocks[0]
            if isinstance(tmp, dict):
                final_text = tmp.get("text", "")
            elif hasattr(tmp, "text"):
                final_text = tmp.text
            else:
                final_text = str(tmp)

    return {
        "skill": skill.name,
        "answer": final_text or "",
    }


def preview_skill(prompt: str) -> dict:
    skills = build_skills(call_calc_agent_tool, call_random_agent_tool)
    skill = select_skill(prompt, skills)

    return {
```

```
        "selected_skill": skill.name,
        "description": skill.description,
        "tool_count": len(skill.tools),
    }


async def main() -> None:
    async for chunk in stream_orchestrator(
        "Flip a coin. And then pick a color shirt for me to wear."
    ):
        print(chunk, end="")


if __name__ == "__main__":
    asyncio.run(main())
```

3) Update `main.py`

Your current `/chat/{prompt}/skills` route has a mismatch because it does not accept `prompt`.

I'd clean that up like this:

```
from fastapi import FastAPI
from fastapi.responses import StreamingResponse

from agents.orch_agent.client import stream_orchestrator, orch, preview_skill

app = FastAPI()


@app.get("/")
def read_root():
    return {"Hello": "World"}


@app.get("/chat/{prompt}")
async def query_chat(prompt: str):
    return await orch(prompt)


@app.get("/chat/stream/{prompt}")
async def query_chat_stream(prompt: str):
    return StreamingResponse(
        stream_orchestrator(prompt),
```

```
        media_type="application/x-ndjson",
    )


@app.get("/chat/{prompt}/skills")
def skills_chat(prompt: str):
    return preview_skill(prompt)
```

What this gives you immediately

For a prompt like:

- `"what is 8 + 12?"`
  the orchestrator selects `math`, loads only the calc tool, and uses a math-specific prompt.

For:

- `"flip a coin"`
  it selects `random`, loads only the random tool.

For:

- `"flip a coin and then add 4 + 5"`
  it selects `general`, loads both tools.

That is exactly the kind of “Skills” behavior you want in a simple version.

What I would not do yet

I would avoid all of this for now:

- skill plugins
- filesystem skill loaders
- YAML skill manifests
- embedding-based skill routing
- LLM-based skill classification
- nested skill execution graphs
- putting skills inside each sub-agent

Those are all possible later, but they would be overkill for this codebase right now.

One small improvement I strongly recommend

Right now the sub-agent clients return:

```
{
    "answer": answer,
    "trace": trace,
}
```

That is fine, but the orchestrator tool wrappers were typed as `-> str`.

So while touching this code, I would definitely normalize the wrappers to return only:

Python Run

That keeps tool behavior clean and predictable.

You still keep the trace logic inside the sub-agent clients for debugging, but the orchestrator stays simple.

The overall design after the change

The flow becomes:

1. FastAPI receives prompt
2. orchestrator selects skill
3. skill limits instructions + tools
4. orchestrator agent runs
5. sub-agent handles domain task
6. response returns with selected skill

So this is still your same project, just with a lightweight skill layer in the right place.

If you want, I can turn this into a **full patch version of the updated files** so you can drop them into your project directly.

yes please

I'm pulling the exact files now so I can give you a drop-in version rather than pseudo-code. I'll keep the patch small and aligned to your current project.

I've got the exact patch path. I'm generating a drop-in updated copy of the project now with one new Skills file and only the smallest orchestrator/API changes.

Thought for 1m 6s >

Done — I made a drop-in updated copy of the project with a lightweight Skills layer.

Files:

- Updated project zip
- New `agents/orch_agent/skills.py`
- Updated `agents/orch_agent/client.py`
- Updated `main.py`
- Patch notes

What changed:

- Added a simple `math`, `random`, and `general` skill selector.
- Kept the calc and random MCP servers unchanged.
- Updated the orchestrator so it picks a skill first, then only exposes the relevant tool set.
- Fixed the `/chat/{prompt}/skills` endpoint so it actually previews the selected skill.

Behavior now:

- Math prompts route to `math`
- Random prompts route to `random`
- Mixed/unclear prompts route to `general`

Example:

- `/chat/what is 8 + 12?`
- `/chat/flip a coin`
- `/chat/flip a coin and add 4 + 5`
- `/chat/what is 8 + 12?/skills`

One important note:

- I also cleaned up the orchestrator tool wrappers so they return just the sub-agent's final answer text instead of the whole `{answer, trace}` dict. That keeps the tool contract simple.

If you want, next I can make the same Skills pattern available to your `deep_client.py` path too, while keeping it just as simple.

fastmcp_agentic_phase1_...
Zip Archive

okay now I'm going to upload the project I've been working on with you. Can you tell me how different it is, architectural wise? I understand that the agents will be different, but I just want to get an idea of how different the core areas are

I'm unpacking this one and mapping its structure against the simpler multi-agent project so I can compare the core architecture directly rather than just guessing from file names.

This one is already clearly more than a toy multi-agent app. The big difference I've confirmed so far is that your newer project has a real host/runtime layer, a persistent MCP client runtime, a session store, and built-in observability as first-class architecture instead of side logic.

Thought for 1m 4s

Yes — it is **meaningfully different architecturally**, but not in a "completely different species" way.

The simplest way to say it is:

your new project is the same core idea, but much more host-centric, modular, and production-oriented.
The earlier project was a **small multi-agent demo**.
This newer one is closer to a **real agent platform skeleton**.

My overall take

They are still related at the core:

- both use **FastMCP tools**
- both have a **central orchestration layer**
- both use an **LLM to decide when to use tools**
- both can route specialized work to domain-specific capabilities

But the **shape of the architecture** is different.

The older project was basically:

- tool servers
- one client per sub-agent
- one top-level orchestrator agent that calls those sub-agents

The newer project is more like:

- independent MCP servers
- one reusable MCP runtime that loads all tool servers
- one orchestrator loop that directly manages tool-calling
- session storage
- skills registry/loader
- observability and benchmarking around the whole system

So the newer one is much less of a "tool demo with multiple agents" and much more of a **backend framework for agentic applications**.

Biggest architectural difference

1. Old project: agent-of-agents

In the simpler project, the top-level orchestrator used wrapper tools like:

- `call_calc_agent_tool(...)`
- `call_random_agent_tool(...)`

Those wrappers called separate sub-agent clients, and those sub-agent clients talked to MCP servers.

So the flow was roughly:

User → orchestrator agent → sub-agent wrapper → sub-agent client → MCP server

That is a layered **agent calling agent** pattern.

2. New project: host + unified tool runtime

In your newer project, `backend/mcp/client.py` creates an `MCPRuntime` that opens sessions to all configured MCP servers and loads their tools into one merged tool list.

Then `backend/agent/orchestrator.py` runs a direct orchestrator loop over those tools.

So the flow is more like:

User → FastAPI host → orchestrator loop → bound MCP tools

That is a **host-centric orchestration** pattern, not really a "sub-agent client per domain" pattern anymore.

That is the biggest architectural shift.

What stayed the same

At the conceptual level, these are still cousins.

Both systems still have:

- a central reasoning layer
- specialist capabilities behind tool boundaries
- an LLM deciding when to invoke those capabilities
- MCP as the contract between the host and remote functionality

So if you asked, "did my app become unrecognizably different?"
I would say **no**.

If you asked, "did it mature from a demo architecture into a framework architecture?"
I would say **yes**.

Core area by core area

Orchestration

Old project

- Orchestration mostly came from LangChain agent composition.
- The orchestrator itself delegated to sub-agent wrappers.
- A lot of the routing logic was implicit inside the agent/tool setup.

New project

- `backend/agent/orchestrator.py` contains a very explicit orchestration loop.
- It manually:
  - builds messages
  - binds tools
  - invokes the LLM

  - extracts tool calls
  - calls tools
  - appends `ToolMessage`s
  - repeats until done

That is a much more controlled orchestration architecture.

So here, the newer project is **substantially more advanced**.

Tool integration

Old project

- Each domain had its own client file:
  - calc client
  - random client
- Each client loaded tools from its own MCP server.
- The orchestrator did not directly manage a general MCP runtime.

New project

- `backend/mcp/client.py` gives you one reusable MCP runtime for all configured servers.
- `backend/mcp/connections.py` builds server connections from config.
- Tools are merged into one common runtime.

That means your new system is much more **scalable and uniform**.

Adding a tool family now feels more like:

1. add an MCP server
2. add it to config
3. optionally add a skill

instead of:

1. make a server
2. make a client
3. make orchestrator wrappers
4. wire agent-to-agent routing

So the newer project is **architecturally cleaner** in this area.

Skills

This is one of the clearest differences.

Old project

- Skills were not really first-class.
- They had to be bolted onto the orchestrator manually.

New project

- Skills are a native architecture layer:
  - `backend/skills/registry.py`
  - `backend/skills/loader.py`
  - `skills/registry.json`
  - `skills/.../skill.md`

This is a real Skills design:

- selection
- loading
- prompt injection
- tool allowlisting

That is much closer to the architecture you've been aiming for in our other chats.

So in the Skills area, the new project is **far ahead** of the earlier one.

Session and memory model

Old project

- Mostly request-driven.
- Not much real session architecture.
- Little to no durable conversation management beyond the immediate flow.

New project

- `backend/agent/sessions.py` has a `ChatSessionStore`.
- History is stored by `session_id`.
- The orchestrator receives prior message history.
- There is locking around access.

That means the newer project actually has a **conversation-state architecture**, not just a single-turn demo style.

That is a major core difference.

Observability and metrics

This is another huge jump.

Old project

- Had custom tracing ideas around tool results and execution trees.
- Useful, but still experimental and localized.

New project

- Observability is built into the backbone:
  - `backend/metrics/collector.py`
  - `backend/metrics/quality.py`
  - request metrics
  - tool-call metrics
  - LLM-call metrics
  - OTEL hooks
  - JSONL metrics
  - benchmark scripts
  - quality scoring for tool choice

So the older project had **trace experiments**.
The new one has **an observability architecture**.

That is a major architectural evolution, not a minor difference.

Configuration and deployment model

Old project

- More hardcoded and local-demo oriented.
- More fixed wiring between components.

New project

- `backend/config.py` centralizes:
  - LLM config
  - MCP server list
  - Skills toggle

    - prompt mode
    - CORS
    - OTEL
    - MLflow
- `backend/main.py` uses lifespan startup to assemble the app.

That means your new project is much more like a **deployable backend service**.

Extensibility model

Old project

- To add a new capability, you were really adding another sub-agent path.

New project

- To add a new capability, you are mostly adding another **tool family/server** and optionally a skill.
- The orchestrator does not need to become another nest of client wrappers.

That is a different extensibility philosophy.

The old project scales by adding **more agents**.
The new project scales by adding **more tools + better routing/skills**.

That is a very important architectural distinction.

So how different are they, really?

I'd put it like this:

Same family

They are absolutely in the same family.

Both are agentic systems using MCP and an LLM orchestrator.

Different maturity level

The newer project is **not just a bigger version** of the old one.
It has changed the architectural center of gravity.

The old one is centered on:

- sub-agents
- wrapper tools
- demo-style delegation

The new one is centered on:

- host/runtime
- direct tool orchestration
- skills
- sessions
- observability
- benchmarking

So the “core areas” are **pretty different in implementation**, even though the overall concept is still related.

My plain-English verdict

If I had to summarize it in one sentence:

Your earlier project is a multi-agent prototype, while your newer project is the beginning of an agent orchestration platform.

Or even more plainly:

- **Old project:** “I have a few specialized agents talking through MCP.”
- **New project:** “I have a host application that manages tools, skills, state, and observability around agent behavior.”

That is a real architectural step up.

The single most important difference to remember

The earlier project thinks in terms of:

“Which agent should I call?”

Your newer project thinks more in terms of:

“Which tools should be exposed for this turn, under which skill, with which telemetry?”

That shift is why the newer one feels much more structured.

Bottom line

They are **similar in concept**, but **quite different in core architecture**.

The newer project is stronger in these core areas:

- orchestration control
- tool runtime design
- skills
- session memory
- observability
- benchmarking
- config-driven extensibility

So if your concern is “did I drift too far from the earlier architecture?” — not really.

If your concern is “is this now a more serious architecture than the earlier one?” — very much yes.

I can also give you a **side-by-side architecture map** of:

- old project components
- new project equivalents
- what was replaced, what stayed, and what is genuinely new.

so talk to me about sub-agents. What exactly do you mean and does my newer project not have those?

I’m checking the newer project’s orchestration files directly so I can answer this in terms of your code, not just general definitions.

Thought for 49s ›

Yeah — this is an important distinction.

When I say **sub-agent**, I mean something more specific than just “a separate capability.”

What a sub-agent usually is

A sub-agent is usually a component that has its own:

- instructions or role
- toolset
- reasoning loop
- often its own LLM call(s)
- bounded responsibility

So a parent/orchestrator does **not** directly do the whole task itself.
Instead, it says something like:

- "math agent, handle this"
- "research agent, handle this"
- "data analyst agent, handle this"

Then that sub-agent reasons and returns a result.

In your earlier small project, you *did* have sub-agents

That older project had a pretty classic sub-agent pattern.

Why?

Because the top-level orchestrator did not directly use the calculator MCP tools itself.
Instead, it called wrappers like:

- calculator sub-agent
- random sub-agent

And each of those had its own client logic and own agent setup.

So the shape was basically:

orchestrator agent → sub-agent → MCP tools

That is why I called it an **agent-of-agents** pattern.

The "calculator" piece was not just a raw tool.
It was a little agent boundary.

In your newer project, not really — at least not in the same sense

Your newer project is much flatter.

It looks more like:

host/orchestrator → unified tool runtime → MCP tools

So now the orchestrator is the main reasoning layer, and it directly decides:

- which tools are visible this turn
- which tool to call
- when to stop

That means the newer project is mostly a **single-agent orchestrator with many tools**, not a true sub-agent system.

The easiest way to think about it

Here's the distinction:

Tool

A tool is a callable function/capability.

Examples in your newer project:

- `add`
- `subtract`
- `count_quoted_chars`
- `list_datasets`
- `run_sql`

These do work, but they do not "think."

Skill

A skill is a routing/instruction layer.

It says things like:

- for this kind of request, use these instructions
- expose these tools
- keep the context smaller

A skill helps the orchestrator behave better, but the skill itself is **not an agent**.

Sub-agent

A sub-agent is a specialized mini-agent.

It has:

- its own prompt/role
- usually its own tool subset
- its own internal reasoning step(s)
- a parent agent calling it like a worker

That is different from both a tool and a skill.

So does your newer project have sub-agents?

Strict answer:

No, not in the explicit architectural sense your earlier project had.

Softer answer:

It has **specialized components**, but most of them are **tools/services**, not sub-agents.

For example:

- your calculator MCP server is not a sub-agent
- your quote counter MCP server is not a sub-agent
- your data analyst MCP server is still mainly a tool server, even though it is richer

Those servers expose capabilities, but they are not independently doing LLM-driven reasoning on behalf of the orchestrator.

So I would call them:

- **specialized tool providers**
- **MCP capability servers**
- **domain-specific tool backends**

Not true sub-agents.

Why your newer project feels agentic anyway

Because the orchestrator itself is doing the “agent” behavior.

Your `backend/agent/orchestrator.py` is the thing that:

- receives the user message
- loads skill instructions
- filters tools
- binds tools to the model
- handles tool calls
- loops until final answer

So the newer project still has an agent — just mostly **one main orchestrator agent**.

That is why it feels agentic even without explicit sub-agents.

A good mental model for your newer project

I’d describe it like this:

- **One main orchestrator agent**
- **Many MCP tools**
- **Optional Skills layer**
- **Session store**
- **Observability around the whole loop**

That is not a multi-sub-agent architecture.
It is a **single-agent tool-orchestration architecture**.

Why this is not a bad thing

This is actually often a cleaner architecture.

Sub-agents can be useful, but they also add:

- more prompt surfaces
- more LLM calls
- more latency
- more tracing complexity
- more handoff bugs
- harder debugging

Your newer project avoids a lot of that by keeping the reasoning centralized.

So compared to your older project, your newer one is less:

- "which agent should I call?"

and more:

- "which tools should this orchestrator expose and use for this turn?"

That is a very normal and often better design.

Where your newer project is *closest* to sub-agents

The closest thing you have right now is probably the combination of:

- **Skill**
- **allowed tools**
- **specialized MCP server**

For example, your `data_analyst` path is close in spirit to a sub-agent because it has:

- a specialized domain
- special instructions
- specialized tools

But because the main orchestrator is still doing the reasoning loop, I would still say it is **skill-scoped tool use**, not a real sub-agent.

What would make it a true sub-agent architecture?

Your newer project would become sub-agent-based if you changed it so that, for example:

- the main orchestrator detects a data analysis request
- then calls a dedicated `DataAnalystAgent`
- that `DataAnalystAgent` has its own prompt, its own tool loop, its own internal LLM calls
- then it returns a result to the main orchestrator

That would be a real sub-agent.

So the flow would become:

main orchestrator → data analyst sub-agent → data tools

instead of:

main orchestrator → data tools directly

That is the key difference.

So the practical answer for your codebase

Your newer project currently has:

- **one main agent**
- **many tools**
- **skills**
- **no strongly defined sub-agents**

Your earlier project had:

- **one orchestrator**
- **multiple subordinate agents**
- **each sub-agent talking to its own tools**

So yes, your newer project is less "sub-agent-y."

My recommendation

For your newer project, I would **not** rush to add sub-agents everywhere.

Your current architecture is actually in a good place.

I'd only introduce sub-agents when one of these becomes true:

- a domain needs its own multi-step reasoning loop
- a domain needs its own prompt that is too large/specialized
- a domain needs isolated behavior from the main orchestrator
- a domain needs its own retry/error-handling strategy
- a domain has enough complexity that "just tools + skills" starts feeling messy

That is when sub-agents start making architectural sense.

Right now, your newer project looks more like a **well-structured orchestrator with skill-scoped tools**, not a **hierarchical multi-agent system**.

If you want, I can sketch what your current project would look like in two versions:

1. **as it is now**
2. **how it would look if you introduced true sub-agents for things like data analyst or quote-counting**